

Labolatorium nr 6

Indeksy do Kluczy Obcych

Paweł Myszka, nr albumu 331720

8 stycznia 2026

1 Wstęp

Celem zadania 6 było napisanie procedury SQL, która automatycznie znajduje wszystkie klucze obce w wybranej bazie danych i tworzy dla nich indeksy (jeśli ich jeszcze nie mają). Procedura powinna być parametryczna i pracować na dowolnej bazie danych.

2 Teoria

2.1 Klucze Obce

Klucz obcy (Foreign Key) to ograniczenie integralności referencyjnej, które:

- Nie pozwala dodać rekordu do tabeli podrzędnej, jeśli odpowiadający mu rekord nie istnieje w tabeli nadrzędnej
- Nie pozwala usunąć rekordu z tabeli nadrzędnej, jeśli istnieją powiazane rekordy w tabeli podrzędnej
- Zapewnia spójność danych w bazie

2.2 Indeksy na Kluczach Obcych

Standardowo założenie klucza obcego w SQL Server **nie powoduje automatycznego utworzenia indeksu**. Jednak:

- **Sprawdzenie klucza obcego** podczas wstawiania/usuwania danych wymaga szybkiego wyszukania rekordu w tabeli nadrzędnej
- **Kwerendy z warunkami WHERE** na kluczach obcych (np. `WHERE id_faktury = 5`) są znacznie szybsze z indeksem
- **Złączenia (JOINy)** korzystają z indeksów dla poprawy wydajności

3 Opis Procedury Z6_INDEKSY_FK

3.1 Parametry

`@NazwaBazy NVARCHAR(100)`

Nazwa bazy danych, w której należy utworzyć indeksy dla kluczy obcych.

3.2 Algorytm

Procedura wykonuje następujące kroki:

1. Sprawdza czy podana baza danych istnieje
2. Wysyła dynamiczny SQL do wybranej bazy
3. Wykonuje cursor po wszystkich kluczach obcych
4. Dla każdego klucza obcego sprawdza czy indeks o nazwie `FKI_TabNadrz__TabPodrz` już istnieje
5. Jeśli indeksu brakuje - tworzy go za pomocą `CREATE INDEX`
6. Wyświetla raport z liczbą utworzonych indeksów

3.3 Nazewnictwo Indeksów

Indeksy są tworzone z następującą konwencją nazewnictwa:

`FKI_{TabelaNadrzedna}__{TabelaPodrzedna}`

Przykład: `FKI_Faktura__PozFa`

4 Kod Procedury

```
CREATE PROCEDURE dbo.Z6_INDEKSY_FK
    @NazwaBazy NVARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON

    DECLARE @tabela_podrz NVARCHAR(128)
    DECLARE @kolumna_fk NVARCHAR(128)
    DECLARE @tabela_nadrz NVARCHAR(128)
    DECLARE @nazwa_indeksu NVARCHAR(128)
    DECLARE @sql NVARCHAR(MAX)
    DECLARE @err_msg NVARCHAR(500)
    DECLARE @licznik INT = 0

    BEGIN TRY
        -- Sprawdzenie czy baza istnieje
        IF NOT EXISTS (SELECT 1 FROM sys.databases WHERE name =
            @NazwaBazy)
```

```

BEGIN
    SET @err_msg = N'Baza danych ' + @NazwaBazy + N' nie
        istnieje!'
    RAISERROR(@err_msg, 16, 1)
END

-- Dynamiczny SQL z kursorem po kluczach obcych
SET @sql = N'
USE [' + @NazwaBazy + N'];

DECLARE @tabela_podrz NVARCHAR(128);
DECLARE @kolumna_fk NVARCHAR(128);
DECLARE @tabela_nadrz NVARCHAR(128);
DECLARE @nazwa_indeksu NVARCHAR(128);
DECLARE @sql_idx NVARCHAR(MAX);

DECLARE fk_cursor CURSOR FOR
    SELECT
        tab_podrz.name AS tabela_podrz,
        col_fk.name AS kolumna_fk,
        tab_nadrz.name AS tabela_nadrz
    FROM sys.foreign_keys fk
    INNER JOIN sys.foreign_key_columns fkc
        ON fk.object_id = fkc.constraint_object_id
    INNER JOIN sys.tables tab_podrz
        ON fkc.parent_object_id = tab_podrz.object_id
    INNER JOIN sys.columns col_fk
        ON fkc.parent_object_id = col_fk.object_id
        AND fkc.parent_column_id = col_fk.column_id
    INNER JOIN sys.tables tab_nadrz
        ON fkc.referenced_object_id = tab_nadrz.object_id
    WHERE NOT EXISTS (
        SELECT 1 FROM sys.indexes idx
        WHERE idx.object_id = tab_podrz.object_id
        AND idx.name = ''FKI_'' + tab_nadrz.name + ''__''
            + tab_podrz.name
        )
    ORDER BY tab_podrz.name, col_fk.name;

OPEN fk_cursor;
FETCH NEXT FROM fk_cursor
    INTO @tabela_podrz, @kolumna_fk, @tabela_nadrz;

WHILE @@FETCH_STATUS = 0
BEGIN
    BEGIN TRY
        SET @nazwa_indeksu =
            N''FKI_'' + @tabela_nadrz + N''__'' +
                @tabela_podrz;

        SET @sql_idx =

```

```

        N''CREATE INDEX ['' + @nazwa_indeksu + N'']
        ON [dbo].['' + @tabela_podrz + N'']
        (['' + @kolumna_fk + N''])'';

        EXEC sp_executesql @sql_idx;

    END TRY
    BEGIN CATCH
        -- Obs uga b d w
    END CATCH

    FETCH NEXT FROM fk_cursor
    INTO @tabela_podrz, @kolumna_fk, @tabela_nadrz;
END

CLOSE fk_cursor;
DEALLOCATE fk_cursor;
,

EXEC sp_executesql @sql

END TRY
BEGIN CATCH
    SET @err_msg = ERROR_MESSAGE()
    PRINT ' B D KRYTYCZNY: ' + @err_msg
    RAISERROR(@err_msg, 16, 1)
END CATCH
END
GO

```

5 Testy

5.1 Test 1: Sprawdzenie Indeksów

```

SELECT name, type_desc, is_primary_key
FROM sys.indexes
WHERE object_id = OBJECT_ID('PozFa')
ORDER BY name

```

Wynik: Indeks FKI_Faktura_PozFa typu NONCLUSTERED został znaleziony.

5.2 Test 2: Lista Wszystkich Indeksów FKI

```

SELECT
    t.name AS tabela,
    i.name AS indeks,
    i.type_desc AS typ,
    c.name AS kolumna
FROM sys.indexes i
INNER JOIN sys.tables t ON i.object_id = t.object_id

```

```

INNER JOIN sys.index_columns ic
    ON i.object_id = ic.object_id AND i.index_id = ic.index_id
INNER JOIN sys.columns c
    ON ic.object_id = c.object_id AND ic.column_id = c.column_id
WHERE i.name LIKE 'FKI_%'
ORDER BY t.name, i.name

```

Wyniki:

- Tabela: DETAIL_TB, Indeks: FKI_MASTER_TB__DETAIL_TB, Kolumna: master_id
- Tabela: PozFa, Indeks: FKI_Faktura__PozFa, Kolumna: id_faktury

5.3 Test 3: Zapytanie z Wymuszonym Indekssem

```

SELECT
    f.id_faktury,
    p.towar,
    'Uzyte indeksu FKI_Faktura__PozFa' AS metoda
FROM Faktura f
JOIN PozFa p WITH (INDEX(FKI_Faktura__PozFa))
    ON (f.id_faktury = p.id_faktury)
ORDER BY f.id_faktury, p.towar

```

Wyniki (4 wiersze):

- id_faktury: 1, towar: Laptop
- id_faktury: 1, towar: Mysz
- id_faktury: 2, towar: Monitor
- id_faktury: 3, towar: Klawiatura

6 Wnioski

1. Procedura Z6_INDEKSY_FK prawidłowo identyfikuje klucze obce w bazie danych
2. Automatycznie tworzy indeksy dla kluczy obcych, które ich nie mają
3. Prawidłowo stosuje konwencje nazewnictwa FKI_TabNadrz__TabPodrz
4. Indeksy są typu NONCLUSTERED i mogą być wykorzystywane w operacjach JOINów
5. Procedura jest parametryczna i może pracować na dowolnej bazie danych
6. Prawidłowa obsługa błędów poprzez BEGIN TRY / BEGIN CATCH